RCSP: Multidimensional Risk-Constrained Skill Planning for Eldercare Robots

***Abstract*— Large language models (LLMs) improve robot task planning, but generated plans can violate safety constraints in complex, long-horizon tasks. This limitation is especially important in eldercare, where older adults are more vulnerable and less tolerant of risk. Existing planners use predefined executable skills as the action interface and select, combine, and order skills from a skill library. We propose Risk-Constrained Skill Planning (RCSP), which represents task risk along seven dimensions using the user state, environment state, skill properties, and execution order. RCSP predicts residual risk, applies dimension-wise risk thresholds, and feeds risk violations back to the LLM for replanning. We evaluate RCSP on contextualized eldercare tasks in VirtualHome and on the SilverAgeCompanion robot platform. RCSP achieves a 70.5% safe task success rate in VirtualHome, compared with 56.5% for SayCan, corresponding to a relative improvement of 24.78%. We also construct a contextualized eldercare task dataset for safe skill planning. The data and code will be released in a designated GitHub repository.**

***Index Terms*— eldercare robots, large language models, skill planning, risk constraints, safe planning, replanning**

I. Introduction

Large language models (LLMs) have been increasingly used to select, combine, and order action units from predefined executable robot skills [1]–[3]. The resulting skill plan provides a high-level planning interface for embodied agents. A skill is an atomic action or motion capability defined at the robot control and execution layer, such as moving, navigating, grasping, delivering, reminding, playing media, or interacting. In these systems, the LLM primarily determines which existing capabilities to invoke and how to organize them, rather than directly generating new low-level robot capabilities [4]–[7].

In eldercare, an executable plan is not necessarily safe for the current user. Older adults are more vulnerable and less tolerant of risk, making safety, personalization, privacy, and acceptability important considerations for assistive robots [8]–[12]. An improper movement, transfer, or medication-related action can cause task failure, health damage, physical injury, privacy leakage, or irreversible consequences. Eldercare skill planning must therefore determine both whether a plan completes the task and whether it is suitable for the current user and environment.

Risk in eldercare tasks is context-dependent. The risk of a robot skill changes with the user's health, mobility, cognitive ability, and care needs. Robot state and environmental conditions also affect risk. Existing assistive-robot studies emphasize the importance of adapting robot behavior to users and considering privacy and interaction requirements [9]–[11]. Skill risks are not independent: an earlier action can change the user or environment state and increase the risk of a later skill. Planning based only on individual skill executability or LLM confidence does not capture risk over the complete execution sequence.

Consider an older adult who feels dizzy and asks for help. GPT-4 may generate a plan that moves to the user, helps the user stand, accompanies the user to the bedroom, retrieves medication, and reminds the user to take it. Helping a dizzy user stand directly can increase health and physical interaction risks. Consecutive movements can also propagate risk from earlier actions to later steps. The system should reject the unsafe actions, identify the violated health and physical interaction constraints, and guide the LLM to revise the plan. A safer plan may ask the user to remain seated, check the user's symptoms and level of consciousness, inspect the surroundings, and contact a caregiver.

Safety constraints trade off against task efficiency. Loose constraints reduce plan rejection and human intervention but can allow actions that are unsuitable for older users. Strict constraints reduce unsafe plans but can cause frequent replanning and unnecessary task termination. Recent LLM-based robot planners have addressed related safety-efficiency trade-offs using explicit task constraints and constrained plan generation [13], [14]. The planner must therefore maintain a safety boundary while using risk information to revise plans.

We propose Risk-Constrained Skill Planning (RCSP), a seven-dimensional risk constraint layer between the LLM planner and the robot executor. RCSP initializes the risk state from the user, robot, and environment context, then predicts plan residual risk in skill execution order. Each risk dimension is compared with a calibrated threshold. A plan is executed only when all dimensions satisfy their thresholds. Otherwise, the system reports the violated dimensions to the LLM, which changes the relevant skills or their execution order and generates a new plan. This design follows the general principle of placing an explicit safety mechanism between LLM-based plan generation and robot execution [13], [14], while modeling eldercare risk as a multidimensional and sequential state.

We evaluate RCSP in VirtualHome, an interactive household simulation environment for embodied agents [15], and on the SilverAgeCompanion robot platform. In VirtualHome, RCSP achieves a 70.5% safe task success rate, compared with 56.5% for SayCan. The relative improvement is 24.78%.

The main contributions are:

- We propose a seven-dimensional risk model for eldercare that incorporates user and environment context, executable skill properties, and execution order into plan residual risk.
- We propose an online RCSP mechanism that constrains plan execution with dimension-wise risk thresholds and reports violated dimensions to the LLM for targeted replanning.
- We design procedures for training risk model parameters and learning dimension-wise risk thresholds from risk annotations and eldercare safety rules.
- We construct an eldercare dataset with 500 base tasks and 2,000 contextualized tasks for risk model training and safe planning evaluation.

II. Related Work

A. LLM-Based Robot Task Planning

LLMs map natural-language instructions to high-level plans composed of predefined executable robot skills. SayCan combines language-model task relevance with skill affordances to satisfy semantic and robot capability constraints [1]. ProgPrompt uses programmatic prompts and predefined action interfaces to generate executable plans [2]. Other approaches use LLMs to generate executable programs or embodied policies, including Code-as-Policies, PaLM-E, and VoxPoser [4], [5], [7]. Grounded household tasks and language-conditioned manipulation have also been studied through ALFRED and CLIPort [16], [17]. ReAct alternates reasoning and action, while Inner Monologue uses environment states, execution outcomes, and human feedback to update subsequent plans [18], [19].

These methods mainly determine whether the next action is reasonable and executable and whether the task can be completed [20]. Safety is often represented implicitly by skill feasibility, task constraints, or execution feedback. More recent methods explicitly introduce safety constraints into LLM-based planning, such as runtime constraint monitoring and constrained plan generation [13], [14]. However, these approaches do not explicitly model risks caused jointly by the user state, environmental conditions, and skill execution order. For eldercare applications, plan and execution success alone are insufficient to determine whether a plan is suitable for execution.

B. Safety-Constrained and Uncertainty-Aware Planning

Existing methods address uncertainty and safety in LLM planning through human intervention and explicit constraints. KnowNo uses conformal prediction to calibrate uncertainty in an LLM planner and requests human assistance when the model cannot make a reliable decision [21]. Safety Chip translates natural-language safety constraints into linear temporal logic (LTL) and uses runtime monitoring to reject violating actions [13]. SELP further introduces equivalence voting and LTL-constrained decoding to generate plans that satisfy user-specified safety constraints [14].

Traditional risk-aware robot planners use chance constraints, conditional value at risk (CVaR), or safety-control constraints to handle collision and state uncertainty [22]–[26]. Classical planning systems also provide formal representations for symbolic task planning and execution constraints [27]–[30]. These methods mainly operate at the trajectory, control, or symbolic planning level. They address model uncertainty, rule violations, or motion safety, but they do not directly model how health, physical interaction, irreversibility, user dependency, environmental uncertainty, execution complexity, and privacy risks evolve across consecutive skills.

C. Assistive Robots for Eldercare and Our Position

Assistive robots for eldercare support companionship, reminders, cognitive assistance, and activities of daily living [8], [31]. Existing studies also examine acceptability, personalization, privacy, and ethics [9]–[12]. However, user-level safety requirements are rarely converted into dynamic constraints for high-level task planning.

RCSP inserts an independent risk constraint layer between the skill plan and robot execution, consistent with safety requirements for personal-care robots [32]. The system initializes a seven-dimensional risk state from the eldercare context, updates residual risk in execution order using skill-specific contributions and weights, and evaluates the plan with learned thresholds for each dimension.

Recent work has also explored combining LLM-based planning with task and motion planning, learning-based plan refinement, and explicit replanning mechanisms [33]–[35]. Other studies investigate vision-language replanning, constraint satisfaction, safe planning, behavior-tree planning, and neuro-symbolic approaches [36]–[41]. These methods motivate the use of feedback and structured constraints in LLM-based planning, but RCSP specifically models eldercare risk as a seven-dimensional, context-dependent, and order-dependent state that can be computed, constrained, and returned to the planner for targeted replanning.

III. Method

Given a natural-language task, an LLM selects, combines, and orders executable skills from the robot's skill library to generate a skill plan. RCSP predicts the plan's seven-dimensional residual risk in skill execution order and compares each dimension with its calibrated threshold. A plan that satisfies all thresholds is executed.

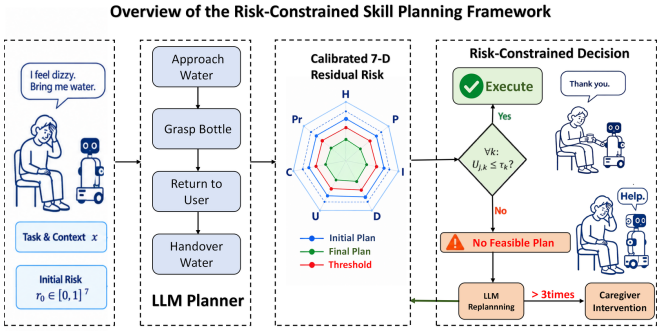


Fig. 1: **Overall structure of the proposed RCSP framework:** The first layer uses an LLM to generate executable skill plans. The second layer evaluates sequential seven-dimensional residual risk. The final layer applies threshold constraints and risk-feedback replanning.

Otherwise, the violated dimensions are returned to the LLM, which changes the selected skills or their order and generates a new plan. The process forms a generation–prediction–constraint–feedback loop. The overall process is shown in Fig. 1.

A. Problem Definition

We study risk-constrained skill planning for eldercare. The planner selects, combines, and orders existing executable robot skills and generates skill plans under risk constraints.

Older users are more vulnerable and have a lower tolerance for risk. The same skill or execution error can therefore produce more serious health and safety consequences. Skill risk changes with the user state, environmental conditions, and execution order. Skill combinations and orderings also affect the overall plan risk.

The planning problem is to balance task completion and eldercare safety. The planning module must assess task risk, constrain the skill plan, and adjust the plan according to the risk assessment. Our objective is autonomous task planning and execution under safety constraints.

B. Risk-Constrained Skill Planning

Skill Definition A skill is a directly callable atomic action or motion capability predefined at the robot control and execution layer, such as moving, navigating, grasping, delivering, reminding, playing media, or interacting. Low-level execution modules provide the skills. The LLM does not generate new skills; it selects, combines, and orders skills from the existing skill library to form a skill plan [1], [2], [4].

Task Understanding and Initial Plan Generation Given a natural-language task l and context c, the LLM identifies the task objective and execution conditions. It selects and organizes executable skills from the skill library to generate a skill plan π [1], [2]. Each planning round generates one current plan. Subsequent plans are generated through replanning after safety checking.

Sequential Risk Assessment The system applies the seven-dimensional risk model defined in Sec. III-C in skill execution order. It combines the initial contextual risk with preceding risk states to compute the current plan's seven-dimensional residual risk.

Safety Constraints and Risk Feedback The system compares the final risk $R_k(\pi)$ in dimension k with the corresponding safety threshold B_k:

$$R_k(\pi) \le B_k, \qquad k = 1, \dots, 7. \tag{1}$$

The system executes the plan when all seven constraints are satisfied. If any dimension exceeds its threshold, the system rejects the plan and reports the violated dimensions to the LLM. The planning module changes the relevant skills or their execution order, generates a new plan, and evaluates it again. This explicit separation between plan generation and safety checking is related to prior safety mechanisms for LLM-driven robot planning [13], [14].

Safe Execution and Online Closed Loop During execution, the system monitors the user, robot, and environment states. If a state change causes the residual risk of the remaining skills to exceed a threshold, the system reevaluates the remaining plan and triggers replanning using the violated dimensions. This use of execution feedback to update subsequent planning is consistent with the broader feedback-based paradigm in embodied LLM planning [19], [33], [36].

C. Sequential Seven-Dimensional Risk Modeling

Initial Contextual Risk Before execution, the system initializes a seven-dimensional risk state $\mathbf{r}_0$ from the user, robot, and environment states and the task context. Each component lies in $[0, 1]$. The dimensions are Health, Physical Interaction, Irreversibility, User Dependency, Environmental Uncertainty, Execution Complexity, and Privacy.

Skill Risk Contribution and Skill-Specific Weights For skill s, the seven-dimensional exposure vector $\mathbf{E}_s$ and mitigation vector $\mathbf{M}_s$ represent its potential risks and mitigation effects. Their normalized difference defines the net skill risk contribution:

$$\mathbf{q}_s = \mathrm{Norm}(\mathbf{E}_s - \mathbf{M}_s). \tag{2}$$

Here, $q_{s,k} > 0$ means that skill s increases risk dimension k, while $q_{s,k} < 0$ means that it reduces that dimension.

Skill-specific weights determine the effect of each skill on each risk dimension:

$$w_{s,k} = \bar{w}_k + m_{s,k}\delta_{s,k}, \qquad w_{s,k} \ge 0. \tag{3}$$

Here, $\bar{w}_k$ is the global weight for dimension k, $\delta_{s,k}$ is the skill-specific offset, and $m_{s,k}$ is an identifiability mask determined by the number of observations.

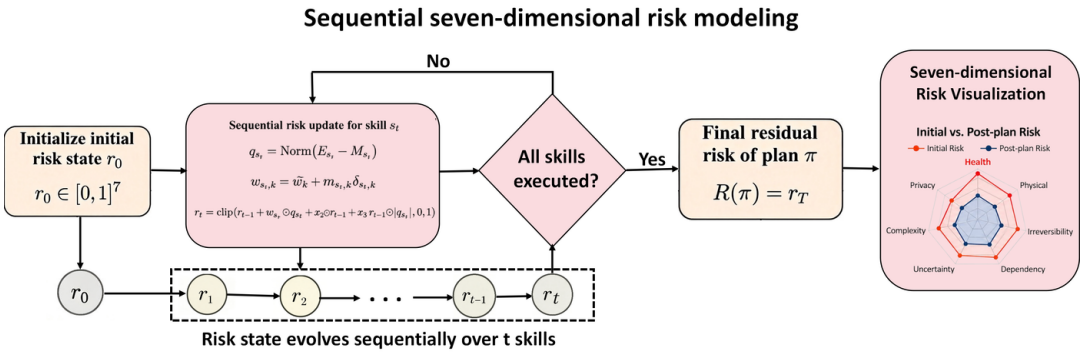


Fig. 2: **Sequential seven-dimensional risk modeling mechanism:** The first layer initializes context-aware risk states from the user, robot, and environment conditions. The second layer updates the risk state using skill-specific contributions and weights. The final layer performs sequential risk updates and outputs the final residual risk vector for safety constraint checking.

For infrequent skills, frequency-aware regularization constrains the skill-specific offsets:

$$\mathcal{L}_{\text{reg}} = \sum_{s \in \mathcal{S}} \sum_{k=1}^{7} \lambda_{s,k} \delta_{s,k}^2, \qquad \lambda_{s,k} = \frac{\lambda_0}{n_s + \epsilon}. \tag{4}$$

Here, $\mathcal{S}$ is the skill set, n_s is the number of observations of skill s, λ_0 is the base regularization strength, and $\epsilon > 0$ is a numerical stability term. Fewer observations produce a larger regularization coefficient and shrink the skill-specific offset toward the global weight.

Sequential Risk Evolution The system updates the risk state for each skill in execution order. When the t-th skill s_t executes, the risk state becomes

$$\begin{aligned} \mathbf{r}_t = \text{clip}\Big(&\mathbf{r}_{t-1} + \mathbf{w}_{s_t} \odot \mathbf{q}_{s_t} + \mathbf{x}_2 \odot \mathbf{r}_{t-1} \\ &+ \mathbf{x}_3 \odot \mathbf{r}_{t-1} \odot |\mathbf{q}_{s_t}|,\, 0, 1\Big). \end{aligned} \tag{5}$$

Here, $\mathbf{r}_{t-1}$ is the residual risk accumulated before the current skill, and $\mathbf{w}_{s_t} \odot \mathbf{q}_{s_t}$ is its base risk contribution. The vector $\mathbf{x}_2$ controls the effect of the previous risk state, and $\mathbf{x}_3$ controls the interaction between previous risk and current skill risk. The operator $\odot$ denotes element-wise multiplication, and $\text{clip}(\cdot, 0, 1)$ limits every dimension to $[0, 1]$.

After the recursion, $\mathbf{r}_T$ is the plan's final seven-dimensional residual risk, denoted by $\mathbf{R}(\pi)$. Plans with the same skills in different orders can therefore have different final risks.

D. Parameter Training and Risk Threshold Learning

Risk Model Parameter Training We train the risk model on 2,000 tasks and their contextual samples. Each sample contains the task and context, seven-dimensional initial risks and risk annotations, skill information, and a safety-validated feasible skill plan with its risk constraints. Human risk annotations supervise model fitting, while the risk constraints of feasible plans are used to estimate

$$\theta = (\bar{\mathbf{w}}, \boldsymbol{\delta}, \mathbf{x}_2, \mathbf{x}_3). \tag{6}$$

Here, $\bar{\mathbf{w}}$ contains the global risk weights, $\boldsymbol{\delta}$ contains skill-specific offsets, $\mathbf{x}_2$ contains the parameters that modulate previous risk, and $\mathbf{x}_3$ contains higher-order interaction parameters.

We optimize the parameters in three stages. First, we learn $\theta_1 = (\bar{\mathbf{w}}, \boldsymbol{\delta})$ for the base skill risks. We then fix θ_1 and learn $\mathbf{x}_2$. Finally, we fix the parameters from the first two stages and learn $\mathbf{x}_3$. The final parameters are

$$\theta^* = (\theta_1^*, \mathbf{x}_2^*, \mathbf{x}_3^*). \tag{7}$$

Risk Threshold Learning After training the risk model, the LLM generates multiple skill plans for each task and context. The system computes each plan's final seven-dimensional risk and labels the plan as Safe or Unsafe using predefined eldercare safety rules. The use of explicit rule-based safety validation is related to prior work that separates task-plan generation from formal or constraint-based safety checking [13], [14].

For dimension k, the risk values of Safe and Unsafe plans are collected in $\mathcal{R}_k^{\text{safe}}$ and $\mathcal{R}_k^{\text{unsafe}}$, with sample counts N_k^{safe} and N_k^{unsafe}. For candidate threshold b, the empirical cumulative distributions are

$$\begin{aligned} \widehat{F}_k^{\text{safe}}(b) &= \frac{1}{N_k^{\text{safe}}} \sum_{r \in \mathcal{R}_k^{\text{safe}}} \mathbb{I}[r \le b], \\ \widehat{F}_k^{\text{unsafe}}(b) &= \frac{1}{N_k^{\text{unsafe}}} \sum_{r \in \mathcal{R}_k^{\text{unsafe}}} \mathbb{I}[r \le b]. \end{aligned} \tag{8}$$

The first term is the proportion of Safe plans accepted by the threshold. The second is the proportion of Unsafe plans incorrectly accepted.

We select the threshold for each dimension with the cost-sensitive empirical loss

$$\widehat{\mathcal{L}}_k(b) = \lambda_{\mathrm{UA}} \widehat{F}_k^{\mathrm{unsafe}}(b) + \lambda_{\mathrm{SR}} \left[1 - \widehat{F}_k^{\mathrm{safe}}(b)\right]. \quad (9)$$

Here, λ_{UA} is the cost of accepting an Unsafe plan, and λ_{SR} is the cost of rejecting a Safe plan. We set $\lambda_{\mathrm{UA}} > \lambda_{\mathrm{SR}}$.

The candidate set $\mathcal{B}_k$ contains 0, 1, and the observed risk values from Safe and Unsafe plans. The selected threshold minimizes the empirical loss. If multiple candidates are optimal, we select the smallest:

$$B_k = \min \arg\min_{b \in \mathcal{B}_k} \widehat{\mathcal{L}}_k(b), \qquad k = 1, \ldots, 7. \quad (10)$$

This procedure produces thresholds $B_1, \ldots, B_7$. The trained model parameters and thresholds remain fixed during online planning.

IV. Experiments

RCSP has three components: sequential seven-dimensional residual risk modeling, dimension-wise risk thresholds, and online replanning from risk feedback. We evaluate them through three research questions. RQ1 compares RCSP with SayCan, Inner Monologue, and KnowNo on overall task execution. RQ2 examines the effects of skill-specific weights, sequential risk updates, and risk feedback. RQ3 evaluates real task execution on SilverAgeCompanion.

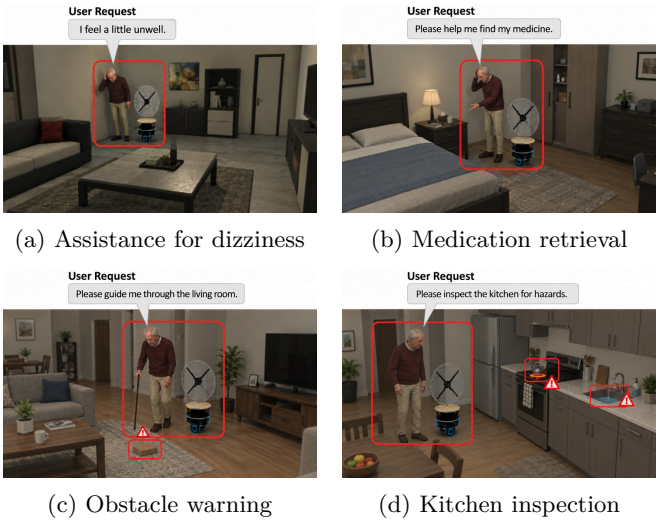
A. Experimental Setup

Environment and Dataset We conduct the simulation experiments in VirtualHome (VH), an interactive household simulation environment for embodied agents [15]. VH provides object states, spatial relations, and action outcomes. We extend its state variables to represent user abilities, care needs, and interaction preferences. The skill library contains 63 executable skills provided by the robot control and execution modules. These skills map high-level natural-language tasks to robot action sequences.

We construct 500 base eldercare tasks in VH. Each task has four context types: a normal setting, changes in the user state, changes in the environment and system, and changes in preferences and historical information. This produces 2,000 contextualized tasks.

Shared Base LLM All simulation experiments use the same base LLM, DeepSeek-V4-Flash. The methods differ only in their planning mechanisms, safety constraints, and risk feedback. All other generation conditions are fixed.

Plan Safety Evaluation We construct an eldercare safety rule set from relevant safety standards, operating guidelines, and task requirements. We convert the rules into checkable conditions using the task objective and context. For each generated skill plan, the system checks the applicable rules against the skill content, execution order, and execution state. A plan is labeled Unsafe if it violates any rule; otherwise, it is labeled Safe. This evaluation is independent of plan generation and risk model training.



(a) Assistance for dizziness (b) Medication retrieval

(c) Obstacle warning (d) Kitchen inspection

Fig. 3: Simulation scenarios for four representative eldercare tasks used in visual analysis and path planning.

Evaluation Metrics Let N be the number of test tasks. Let C_i indicate whether task i is completed, S_i indicate whether its execution satisfies the safety rules, and n_i denote the number of replanning rounds. The task completion rate (TCR) is

$$\mathrm{TCR} = \frac{1}{N} \sum_{i=1}^{N} C_i. \quad (11)$$

The safe task success rate (STSR) is

$$\mathrm{STSR} = \frac{1}{N} \sum_{i=1}^{N} C_i S_i. \quad (12)$$

The average number of replanning rounds (AR) is

$$\mathrm{AR} = \frac{1}{N} \sum_{i=1}^{N} n_i. \quad (13)$$

B. Simulation Experiments

We compare RCSP with SayCan, Inner Monologue, and KnowNo on eldercare tasks. SayCan combines task relevance with skill affordances to select the next action [1]. Inner Monologue converts environment states and execution outcomes into language feedback for subsequent planning [19]. KnowNo uses uncertainty estimation to decide whether to continue autonomous execution or request human assistance [21].

Table I shows that RCSP achieves a TCR of 79.5% and an STSR of 70.5% in VirtualHome. SayCan achieves 71.0% and 56.5%, respectively. Inner Monologue achieves 66.5% and 48.0%, while KnowNo achieves 73.0% and 52.5%. The STSR improvement indicates that the risk constraints reduce high-risk plan execution while retaining task completion.

TABLE I: Performance of different methods in VH

Method	TCR↑	STSR↑	AR
RCSP	79.5%	70.5%	0.91
SayCan	71.0%	56.5%	0
Inner Monologue	66.5%	48.0%	1.1
KnowNo	73.0%	52.5%	0

Risk feedback adjusts skill selection and execution order using violations from the previous plan. Replanning therefore changes the planning decision rather than repeating the initial generation. This process enables some initially unsafe tasks to be corrected.

The difference in replanning rounds reflects the planning mechanisms implemented by each method. SayCan and KnowNo do not perform iterative plan regeneration in our experimental protocol; therefore, their average number of replanning rounds is zero. Inner Monologue uses execution feedback to update the subsequent plan, resulting in an average of 1.1 replanning rounds. RCSP performs risk-based replanning only when the generated plan violates one or more risk constraints, leading to an average of 0.91 replanning rounds.

C. Ablation Study

The ablation study evaluates skill-specific weights, preceding risk states, and plan replanning. The full method is the reference. All configurations use the same tasks, contexts, training split, base LLM, generation budget, and execution budget.

Shared Skill Risk Weights All skills use the same seven-dimensional risk weights. Skill-specific weights are removed, while skill risk contributions and risk accumulation remain unchanged.

Without Sequential Risk Updates The risk of the current skill is computed without the risk accumulated by preceding skills. Only the current skill's risk contribution is used.

Without Plan Replanning The initial plan is still checked. If it fails the check, the system generates no new plan and terminates the task according to the predefined policy.

With shared skill risk weights, TCR and STSR are 67.0% and 58.0%, respectively. Without sequential risk updates, they are 68.5% and 59.5%. Removing plan replanning reduces TCR by 19.5 percentage points. It also yields the lowest STSR among all ablated configurations (55.5%), below both the shared-weight (58.0%) and the no-sequential-update (59.5%) variants, indicating that keeping replanning active is more beneficial than degrading the risk model while retaining replanning.

D. Real-Robot Experiments

We conduct the real-robot experiments on the SilverAgeCompanion platform. A Raspberry Pi 5 coordinates the system components and converts plans into skills. Skill outcomes and perceived states are returned to the planning module for subsequent replanning.

TABLE II: Results of the three component ablations

Configuration	TCR↑	STSR↑	AR
Full method	79.5%	70.5%	0.91
Shared skill risk weights	67.0%	58.0%	1.23
Without plan replanning	60.0%	55.5%	0
Without sequential risk updates	68.5%	59.5%	1.19

TABLE III: Real-robot results in different scenarios

Scenario	Method	TCR↑	STSR↑	AR
Kitchen inspection	GPT-5.5	65%	55%	0
	GLM-5.2	60%	45%	0
	DeepSeek-V4-Flash (RCSP)	65%	60%	1.65
Medication retrieval	GPT-5.5	65%	60%	0
	GLM-5.2	65%	50%	0
	DeepSeek-V4-Flash (RCSP)	75%	60%	1.30
Obstacle warning	GPT-5.5	65%	55%	0
	GLM-5.2	60%	50%	0
	DeepSeek-V4-Flash (RCSP)	80%	70%	1.05
Overall	GPT-5.5	65.0%	56.7%	0
	GLM-5.2	61.7%	48.3%	0
	DeepSeek-V4-Flash (RCSP)	73.3%	63.3%	1.33

We compare GPT-5.5, GLM-5.2, and DeepSeek-V4-Flash with RCSP. The scenarios are kitchen inspection, medication retrieval, and obstacle warning. Each method runs 20 trials per scenario, for 60 trials per method.

Table III reports the results. RCSP achieves an overall TCR of 73.3% and an overall STSR of 63.3%. GPT-5.5 achieves 65.0% and 56.7%, while GLM-5.2 achieves 61.7% and 48.3%, respectively. RCSP is therefore 8.3 and 11.6 percentage points higher in TCR and 6.6 and 15.0 percentage points higher in STSR than GPT-5.5 and GLM-5.2, respectively.

RCSP averages 1.65, 1.30, and 1.05 replanning rounds for kitchen inspection, medication retrieval, and obstacle warning, respectively. The overall average is 1.33. Fewer replanning rounds are required for less complex tasks, showing that RCSP adjusts plans according to risk constraints during execution.

V. Conclusion and Future Work

We propose Risk-Constrained Skill Planning (RCSP) for eldercare robots. RCSP inserts a multidimensional risk constraint layer between the LLM planner and the robot execution layer. It initializes a seven-dimensional risk state from the user, robot, and environment context, then updates residual risk in skill execution order using skill-specific contributions, risk weights, and preceding risk states. The system evaluates each plan with learned thresholds for the seven risk dimensions. When a dimension exceeds its threshold, the system returns the violation to the LLM and replans the relevant skills or their execution order.

We evaluate RCSP on contextualized eldercare tasks in VirtualHome and on the SilverAgeCompanion platform.

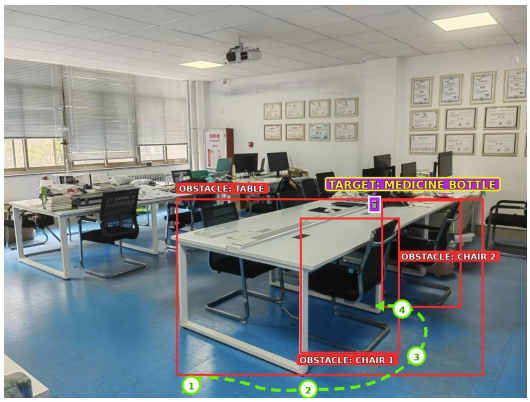


Fig. 4: Context-aware risk-avoidance planning in a robot–older-adult dialogue scenario.



Fig. 5: Real-world experimental scenes on the SilverAge-Companion platform.

RCSP achieves higher safe task success rates than the evaluated baselines while retaining task completion capability. Risk feedback also corrects some initially unsafe plans within a limited number of replanning rounds. The ablation results support the use of skill-specific risk modeling, sequential risk updates, and risk-driven replanning.

Future work will evaluate cross-scenario and cross-platform generalization in additional homes, user states, and long-tail skills. We will also study personalized and time-varying risk thresholds that adapt to user abilities, preferences, and state changes. Finally, we will connect high-level skill risk planning with perception uncertainty, motion planning, and low-level safety control to extend risk management from task planning to physical execution.

References

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Julian, D. Kalashnikov, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," in *Proceedings of the Conference on Robot Learning*, vol. 205, 2022, pp. 287–318.

[2] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "Progprompt: program generation for situated robot task planning using large language models," *Autonomous Robots*, vol. 47, pp. 999–1012, 2023.

[3] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *Proceedings of the 39th International Conference on Machine Learning*, vol. 162, 2022, pp. 9118–9147.

[4] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proceedings of the 2023 IEEE International Conference on Robotics and Automation*, 2023.

[5] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, "Palm-e: An embodied multimodal language model," in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, 2023, pp. 8469–8488.

[6] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of the 7th Conference on Robot Learning*, vol. 229, 2023, pp. 2165–2183.

[7] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "Voxposer: Composable 3d value maps for robotic manipulation with language models," in *Proceedings of the 7th Conference on Robot Learning*, vol. 229, 2023, pp. 540–562.

[8] K. Wada, T. Shibata, T. Saito, and K. Tanie, "Effects of robot-assisted activity for elderly people and nurses at a day service center," *Proceedings of the IEEE*, vol. 92, no. 11, pp. 1780–1788, 2004.

[9] M. Heerink, B. Kröse, V. Evers, and B. Wielinga, "Assessing acceptance of assistive social agent technology by older adults: The almere model," *International Journal of Social Robotics*, vol. 2, pp. 361–375, 2010.

[10] E. Broadbent, R. Stafford, and B. MacDonald, "Acceptance of healthcare robots for the older population: Review and future directions," *International Journal of Social Robotics*, vol. 1, no. 4, pp. 319–330, 2009.

[11] K. Dautenhahn, "Socially intelligent robots: Dimensions of human-robot interaction," *Philosophical Transactions of the Royal Society B: Biological Sciences*, vol. 362, no. 1480, pp. 679–704, 2007.

[12] S. Sabanovic, "Robots in society, society in robots: Mutual shaping of society and technology as a framework for social robot design," *International Journal of Social Robotics*, vol. 2, no. 4, pp. 439–450, 2010.

[13] Z. Yang, S. S. Raman, A. Shah, and S. Tellex, "Plug in the safety chip: Enforcing constraints for llm-driven robot agents," in *2024 IEEE International Conference on Robotics and Automation*, 2024.

[14] Y. Wu, Z. Xiong, Y. Hu, S. S. Iyengar, N. Jiang, A. Bera, L. Tan, and S. Jagannathan, "Selp: Generating safe and efficient task plans for robot agents with large language models," in *2025 IEEE International Conference on Robotics and Automation*, 2025.

[15] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba, "Virtualhome: Simulating household activities via programs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8494–8502.

[16] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han,

R. Mottaghi, L. Zettlemoyer, and D. Fox, "Alfred: A benchmark for interpreting grounded instructions for everyday tasks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 10 740–10 749.

[17] M. Shridhar, L. Manuelli, and D. Fox, "Cliport: What and where pathways for robotic manipulation," in *Proceedings of the 5th Conference on Robot Learning*, vol. 164, 2021, pp. 894–906.

[18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[19] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning*, vol. 205, 2023, pp. 1769–1782.

[20] A. Parisi, Y. Zhao, and N. Fiedel, "Talm: Tool augmented language models," 2022.

[21] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, Z. Xu, D. Sadigh, A. Zeng, and A. Majumdar, "Robots that ask for help: Uncertainty alignment for large language model planners," in *Proceedings of the 7th Conference on Robot Learning*, vol. 229, 2023, pp. 661–682.

[22] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Perez, "Integrated task and motion planning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 265–293, 2021.

[23] Y. Chow, A. Tamar, S. Mannor, and M. Pavone, "Risk-sensitive and robust decision-making: A cvar optimization approach," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[24] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 39–64, 2019.

[25] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[26] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.

[27] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.

[28] M. Ghallab, A. Howe, C. Knoblock, D. McDermott, A. Ram, M. Veloso, D. Weld, and D. Wilkins, "Pddl—the planning domain definition language," Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003/DCS TR-1165, 1998.

[29] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.

[30] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "Stomp: Stochastic trajectory optimization for motion planning," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 4569–4574.

[31] D. Feil-Seifer and M. J. Mataric, "Defining socially assistive robotics," in *Proceedings of the 2005 IEEE 9th International Conference on Rehabilitation Robotics*, 2005, pp. 465–468.

[32] *Robots and Robotic Devices – Safety Requirements for Personal Care Robots*, International Organization for Standardization Std. ISO 13 482:2014, 2014.

[33] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, "Autotamp: Autoregressive task and motion planning with llms as translators and checkers," in *2024 IEEE International Conference on Robotics and Automation*, 2024, pp. 6695–6702.

[34] M. Dalal, T. Chiruvolu, D. S. Chaplot, and R. Salakhutdinov, "Plan-seq-learn: Language model guided rl for solving long horizon robotics tasks," in *International Conference on Learning Representations*, 2024.

[35] S. Izquierdo-Badiola, G. Canal, C. Rizzo, and G. Alenyà, "Plancollabnl: Leveraging large language models for adaptive plan generation in human-robot collaboration," in *2024 IEEE International Conference on Robotics and Automation*, 2024, pp. 17 344–17 350.

[36] A. Mei, G.-N. Zhu, H. Zhang, and Z. Gan, "Replanvlm: Replanning robotic tasks with visual language models," *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 10 201–10 208, 2024.

[37] A. Curtis, N. Kumar, J. Cao, T. Lozano-Perez, and L. P. Kaelbling, "Trust the proc3s: Solving long-horizon robotics problems with llms and constraint satisfaction," in *Conference on Robot Learning*, vol. 270, 2024, pp. 1362–1383.

[38] S. Li, F. Liu, L. Cui, J. Lu, Q. Xiao, X. Yang, P. Liu, K. Sun, Z. Ma, and X. Wang, "Safe planner: Empowering safety awareness in large pre-trained models for robot task planning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 14, 2025, pp. 14 619–14 627.

[39] J. Ao, F. Wu, Y. Wu, A. Swikir, and S. Haddadin, "Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning," in *2025 IEEE International Conference on Robotics and Automation*, 2025, pp. 1233–1239.

[40] M. Kwon, Y. Kim, and Y. J. Kim, "Fast and accurate task planning using neuro-symbolic language models and multi-level goal decomposition," in *2025 IEEE International Conference on Robotics and Automation*, 2025, pp. 16 195–16 201.

[41] A. A. Khan, M. Andrev, M. A. Murtaza, S. Aguilera, R. Zhang, J. Ding, S. Hutchinson, and A. Anwar, "Safety aware task planning via large language models in robotics," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2025, pp. 21 024–21 031.